Lab Assignment 3

What:

In this assignment you are to write a C or Fortran program that preforms a parallel matrix-vector product:

1. Login to prospero via ssh (`ssh <yourusername>@prospero.cs.wit.edu`)
2. Use MPI to parallelize the **matrix-vector product** for a very large matrix.
3. Create a makefile to properly compile your code.
4. Do **NOT** run the program on the login node, create a batch script and submit to the cluster.
5. Verify that your code produces the correct result vector.
6. Time your code with a variety of ranks.
7. Create a `PDF` file with a document that includes the required information from the expected result section below.
8. Leave the PDF file in the same directory on prospero where your completed assignment is.

Using large matrix dimensions is important for this assignment. Find a size that takes 5-10 seconds to run with one rank. As a result, the timings you compute will be less influenced by small tasks running on the compute node (i.e. always do timings with a suitably large problem).

Why:

Collective communication routines are the most common functions that you will call when using MPI. In this assignment you will distribute a 2D structure across multiple nodes. This is a very common action in many problems including: image processing, grid-based simulation, linear algebra, physical and biological simulations, etc. In the future, you will be using collective communication almost exclusively to distribute your problems among nodes.

How:

Each rank will be doing a smaller version of the full matrix-vector multiplication. This means that each rank will have a subset of the number of rows of the full matrix and the entire vector. The result will be a small array (equal to the number of rows in the submatrix) that will need to be gathered at the end of the calculation. Your communication pattern should be as follows:

- Broadcast the full vector to all ranks.
- Scatter the rows of the original matrix to the ranks. Remember, using scatter requires that you evenly distribute the matrix rows to each rank (Unless you want to use `Scatterv` and `Gatherv`, which is not required for this assignment).
- Use `MPI_Gather` to combine the small result arrays back into one large array on rank `0`.

`MPI_Gather()` is the reverse of the `MPI_Scatter()`. It takes the same arguments but uses them in a different way. For example, the `&sendbuffer` represents the smaller array that will be gathered and the `&recvbuffer` is the final, large array that all the small arrays will be gathered into. Ensure that the arrays you create are allocated to the correct size.

Expected Result:

Answer/Do the following:

Plot (using whatever software you want) the number of ranks vs. the time it takes to do the calculation (excluding communication time).

Plot the communication time for all communication that you perform vs. the dimension of the matrix. Use timers around each MPI communication call and sum them.

Generally speaking, a Floating Point Operation is a **single multiply or a single addition** (although this varies based on hardware capabilities). Use this knowledge to compute the number of floating point operations that take place during the calculation of the matrix-vector product for a fixed, large, matrix size (Look at how many additions and multiplications take place in your loops). Then, using the calculation time that you found earlier, compute the Floating Point Operators per Second (FLOPS) that your application can achieve, for the fixed size matrix, versus the number of ranks. Create a table and include it in your PDF.

Overall, you will have two plots and a table.